Rockchip音频算法效果调试指南

文件标识：RK-KF-SF-XXX

发布版本：V1.0.0

日期：2024-08-12

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文提供一个音频效果调参指导文档，以帮助开发人员在Rockchip平台上更好地完成音频效果调试。

产品版本

芯片名称	内核版本
全系列	通用

读者对象

本文档（本指南）主要适用于以下工程师:

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	Xing Zheng	2024-08-12	初始版本

目录

Rockchip音频算法效果调试指南

1. 概述
    1.1 功能描述
2. 准备工作
    2.1 明确当前的产品类型和规格指标
    2.2 音频算法测试相关知识点介绍
    2.3 音频算法测试场景
    2.4 音频通路测试
        2.4.1 音频播放最大不失真信号测试
        2.4.2 音频扫频录放测试
        2.4.3 安防类产品音频算法调试

1. 概述

1.1 功能描述

目前，随着电子产品技术不断地更新迭代，用户希望产品的音频特性能适用于更多的应用场景：在录音场景下，能获以及更为清晰、真实的录音效果；播放场景下，能获得饱满、舒适的听感。这就需要算法的介入与加持，辅助前端硬件的可靠数据通路，适当地调节算法参数，以达到用户不同的听感需求。

2. 准备工作

在面临音频调参需求时，绝大部分开发人员都习惯对着音频算法接口和参数盲调一通。其实，算法调试仅仅是在整个音频效果通路的末端起到辅助作用，真正需要关注的，往往是**算法前端的声学结构、以及硬件器件指标等环节**。只有算法前端获取的信号不失真，算法才能真正有效地发挥它的作用。

因此，当前的产品类型和规格指标，可以在调参之前先明确好，以取得事半功倍的效果。

2.1 明确当前的产品类型和规格指标

首先，开发人员要了解当前的产品类型以及对音频效果的需求。比如：使用几个麦克风，几个喇叭播放？产品类型是IPC，还是会议拾音系统等？是否有双向通话需求？通常IPC类产品不会需要超过2个麦克风拾音，而会议系统要求的麦克风数量较多，若是多麦克风场景下的麦克阵列类型（如4MIC的拓扑，是环麦还是线麦）。

其次，确定好产品设计要求的拾音距离范围（如3米到10米），以及播放响度，以下信息，可通过器件手册查询：

- 麦克风是模拟麦（驻极体居多）还是数字麦？通常模拟麦对远场拾音会有更好的效果
- 麦克风的SNR（通常不低于65dB），灵敏度，以模拟麦为例，5米以上远场拾音，不低于-32dBV
- 喇叭播放输出的功率和负载，如2W@8ohm，
- 产品结构设计相关图片信息，如MIC孔径，还有和SPK的位置、距离、分布等
- 使用硬件回采还是软件回采。目前以软件回采应用居多

接着，还需要确定产品使用的录放的采样率，比如通常为16kHz或者8kHz。若需要使用AI降噪，或者哭声检测的场景，需要使用16kHz及以上的采样率。

2.2 音频算法测试相关知识点介绍

用户大多数需要开启音频算法，除了消除环境噪声以外，还有就是在通话场景下的回声消除AEC（Acoustic Echo Cancellation），这个框图描述了经典回声消除算法在工作时的数据流向和处理效果：

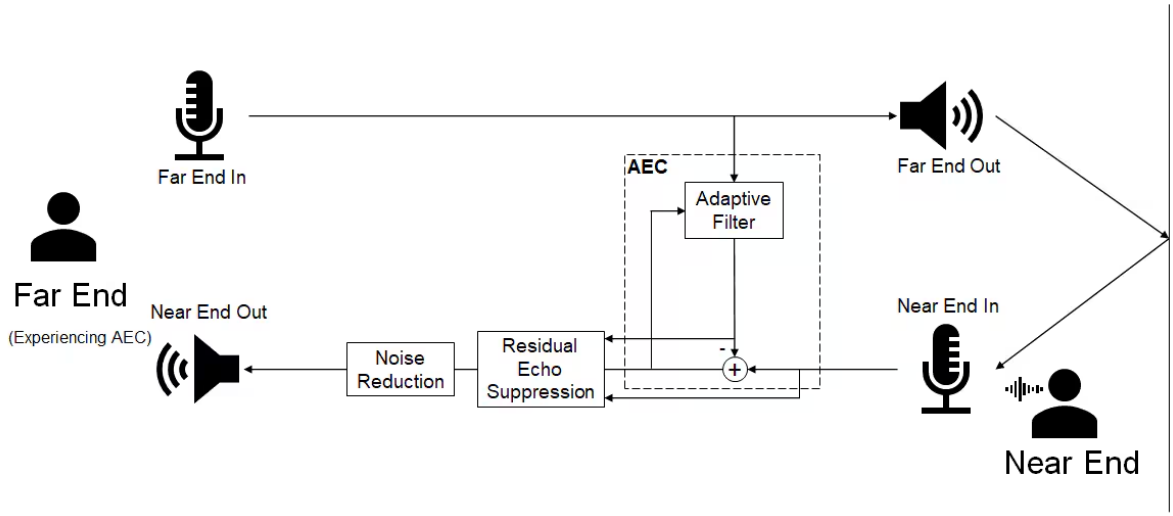


图 1 回声消除工作流

其中，有几个名词术语需要确定：

- 近端（Near End）：以待测机器的角度看，比如主讲人对着机器的MIC说话，我们称之为“近端输入的人声”，接着，这说话声，通过网络传输到远端设备播放出来。
- 远端（Far End）：指与待测机器通讯的远端设备，比如测试人员对着远端设备的MIC说话，通过网络传输到待测机器播放出来。同时，这部分数据也通过回采（不论是软件回采还是硬件回采）的方式，存储在待测机器的另一个录音通道里，作为回声消除的参考信号。

2.3 音频算法测试场景

用户通话对讲时通常会遇到以下使用场景：

- **近端不说话，远端说话**

  在这种情况下，系统处于**远端单讲**状态，远端声音通过近端扬声器播放，再被麦克风采集回去（麦克风输入包含回声和噪声）。这个通常是音频通路测试完以后，音频效果调参最先测试及关注的。测试人员可以在独立的空间内，尽可能使用**自然人声**向远端设备（如手机或另一台对讲设备）说话，然后用远端设备听是否有自己的说话声。如果没有，说明远端单讲下回声抑制的效果较为理想；反之，会有残留回声从设备再回传回来，需要进一步调参优化。

- **近端说话，远端不说话**

  在这种情况下，系统处于**近端单讲**状态，远端没有说话（本机扬声器没有声音），那么也就不存在回声，此时实际相当于麦克风采集到的只有近端主讲人声和环境声。

- **近端和远端同时说话**

  在这种情况下，系统处于**双讲**状态，该状态下自适应滤波器容易发散，导致回声消除力度减小。因此，这时候更需要关注回采参考信号和

  因此，在测试回声消除效果的时候，需要以上三个情况同时测试。

2.4 音频通路测试

在算法调试之前，我们需要对音频通路做测试，调整合适的录放增益，将最佳的电信号数据提供给算法，才可能获得更理想的处理效果。

2.4.1 音频播放最大不失真信号测试

如果一款产品有通话对讲的需求，我们应该先明确喇叭的播放功率和负载，并做好播放最大不失真信号的增益标定。否则，喇叭的失真会引起严重的漏回声异常。

所以，我们可以播放一个0dBFS 1kHz正弦波，然后通过示波器测量功率放大器（PA）输出或者喇叭两端的电压幅值，以确定实际输出的最大不失真电压是否满足产品定位要求。但是，由于PA通常是用ClassD驱动放大，直接测量PA输出或者喇叭两端，会包含几百kHz的高频载波信号。因此，如下图所示，我们可以在喇叭两端分别串上RC电阻形成一阶滤波电路，阻容配比可以为500ohm/10nF，截至频率约为32kHz，既可以滤除高频载波信号，也较为接近人耳可听频率范围上限20kHz。

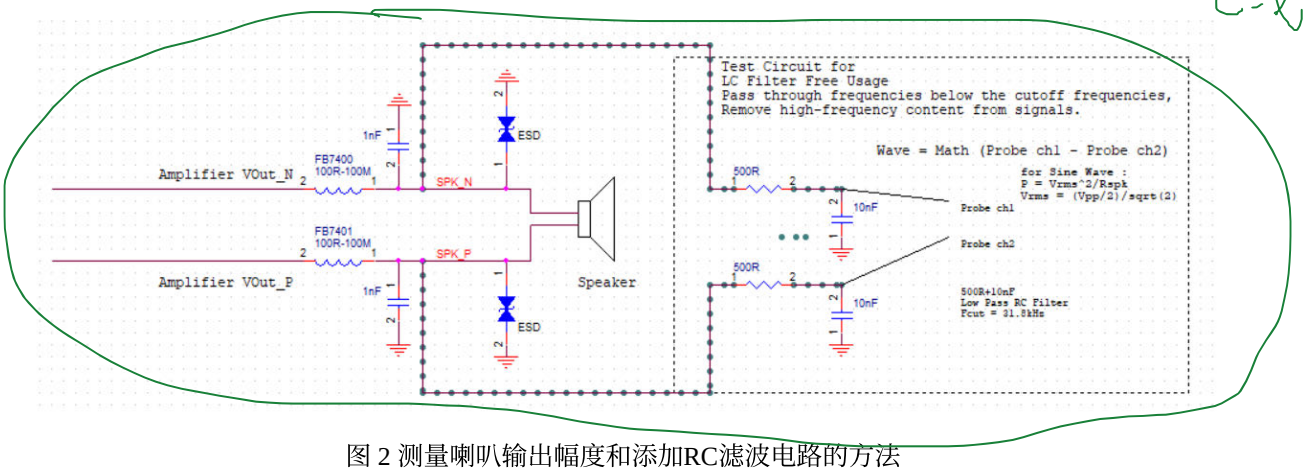


图 2 测量喇叭输出幅度和添加RC滤波电路的方法

播放0dBFS 1kHz正弦波可以使用如下命令，注意播放的是原始数据，不能开启AO VQE算法：

```
rk_mpi_ao_test -i /data/sine1000_16khz_2ch.wav --sound_card_name=hw:0,0 --
device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2
```

如果有tinyalsa或者alsa-utils，也可以使用如下命令播放测试：

```
tinyplay /data/sine1000_16khz_2ch.wav -r 16000 -c 2 -b 2 -D 0
```

```
aplay -Dhw:0,0 /data/sine1000_16khz_2ch.wav -r 16000 -c 2 -f S16_LE
```

经过一番播放增益调整，我们预期能在喇叭端测量到如下信号：



图 3 实测RC滤波后0dBFS 1kHz正弦波最大不失真电压

其中黄色和绿色的波形分别为喇叭的P端和N端，粉色的是示波器经过math sub运算获得的差分电压幅度。如果一个产品设计喇叭的额定功率最高为1.5W@8ohm，那么从粉色的峰-峰值电压9.8V可以得到：

$$P = Vrms^2/R = ((Vpp/2)/\sqrt{2})^2/R = ((9.8/2)/\sqrt{2})^2/8 \approx 1.5W$$

实测功率数值上刚好满足设计要求的1.5W。

2.4.2 音频扫频录放测试

以RV1106/RV1103产品为例，在算法调参之前，我们还需要对音频通路进行测试。通常我们可以播放一个0dBFS的扫频音源，然后适当调节当前的Codec硬件ADC/DAC增益，使其录音到的扫频信号尽可能为最大不失真幅度，即可标定好当前的Codec硬件增益。

首先第一步，先配置好回采通路。以RV1106/RV1103平台举例，可以执行以下命令：

```
rk_mpi_amix_test --control "I2STDM Digital Loopback Mode" --value "Mode2"
```

或者tinyalsa的tinymix：

```
tinymix set "I2STDM Digital Loopback Mode" "Mode2"
```

使能SoC内部数字回采功能。具体ACodec和回采通路细节，请参考《Rockchip_Developer_Guide_Linux_RV1106_ACodec_CN.pdf》。

安防类产品，基本上使用Rockit框架。因为rk_mpi_ai_test默认输出的/tmp/cap_out.pcm有1MB大小的限制，如果需要长时间录音，可以执行以下命令，将原始音频数据导出为/tmp/dbg_capin.pcm：

```
export rt_debug_cap_file=/tmp/dbg_capin.pcm
```

然后执行不开VQE算法的播放命令：

```
rk_mpi_ai_test --sound_card_name=hw:0,0 --device_rate 16000 --device_ch 2 --
out_rate 16000 --out_ch 2 --output /tmp
```

同时，还需要本机播放一个0dBFS的扫频文件：

```
rk_mpi_ao_test -i /data/sweep_16k_20_8000_16b_2ch.wav --sound_card_name=hw:0,0 --
device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2
```

这样，可以获算法处理前的原始录音数据：

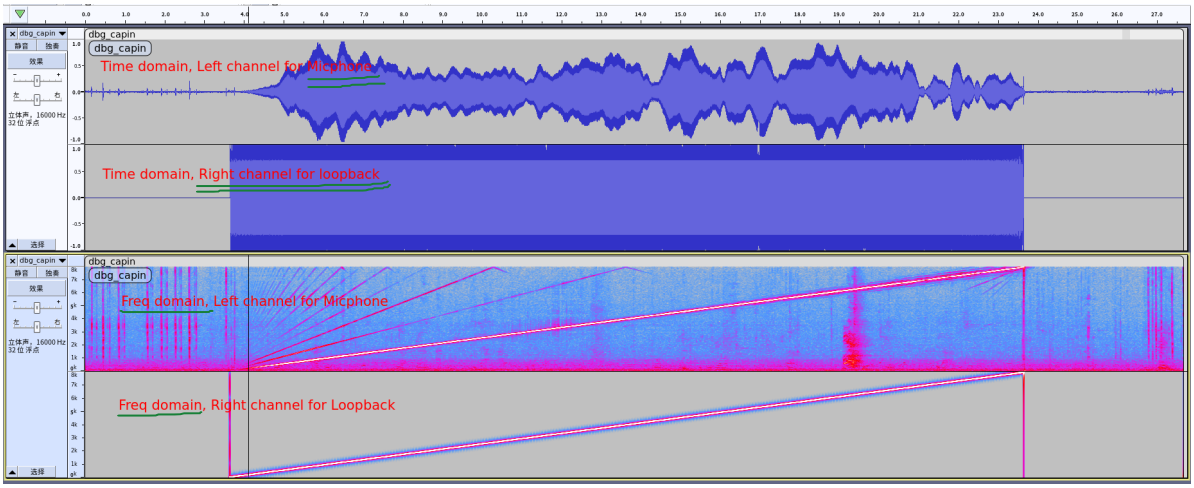


图 4 自录自播扫频测试

如上图可以看到，录到的音频数据是2ch的，并分别显示了2ch的时域和频域的状态。左声道为麦克风实时采集的数据，右声道为回采到刚才播放时的音频数据，且数字回采的PCM和播放出去的PCM内容是一致的。

如果喇叭增益太大，播放0dBFS扫频时可能会测到MIC录到的数据会有截福异常：

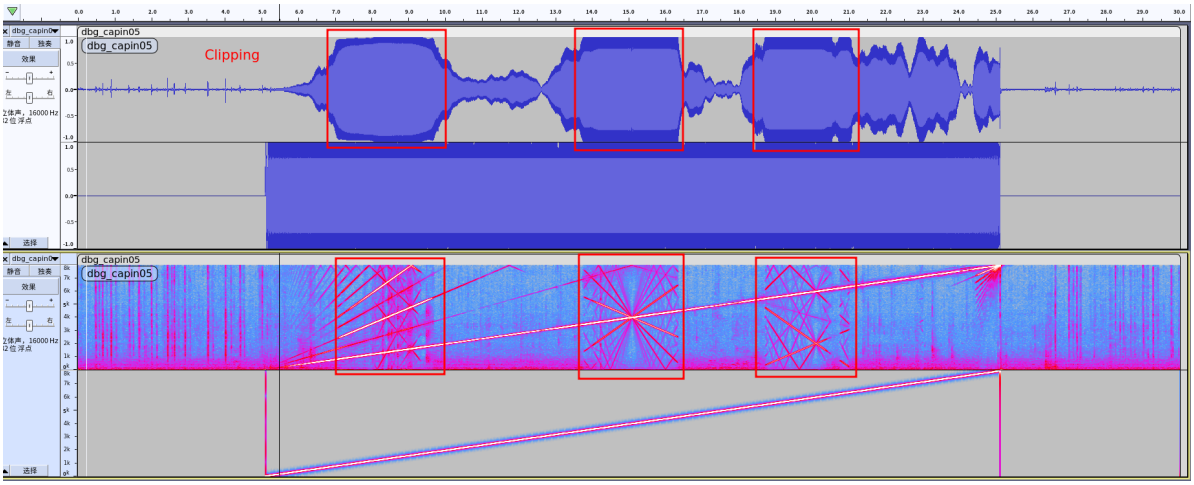


图 5 自录自播扫频测试截福异常

可以将时间轴放大，看到上图红框的部分有明显的截福失真。失真的信号对于后端算法的输入来说是不可逆的异常，会影响算法处理的结果。

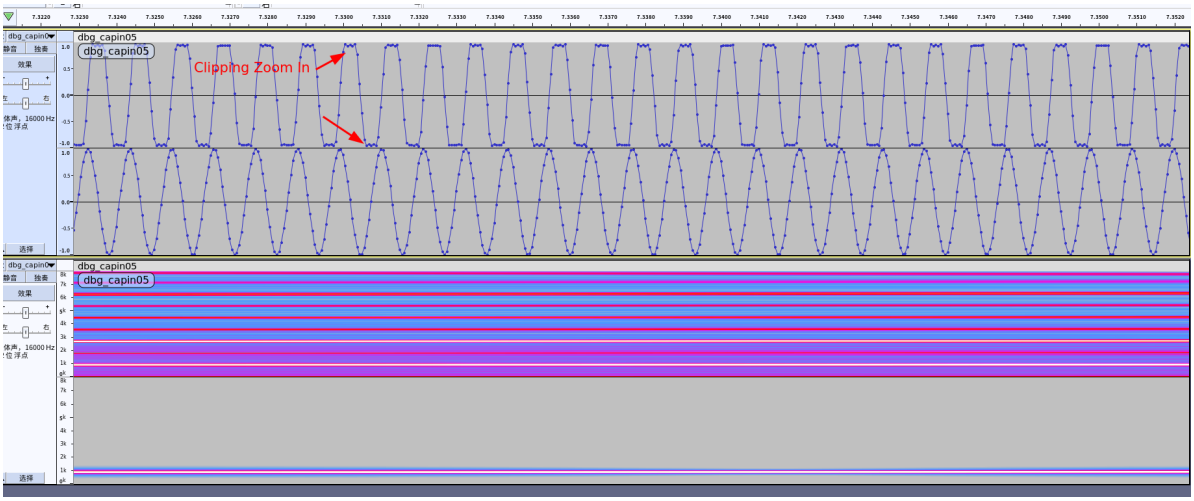


图 6 自录自播扫频测试截福异常示意图

因此，如果有在此情况下遇到录音截福失真，建议先调小前端的codec模拟增益。可以先获取RV1106 ACodec当前的ADC ALC增益：

```
# rk_mpi_amix_test --control "ADC ALC Left Volume"
cmd parse result:
sound control id      : 0
control name          : ADC ALC Left Volume
control value         : (null)
list controls         : 0
list contents         : 0
Get current control value: 22
List the control values:
22 (range 0->31)#
```

可以看到，当前"ADC ALC Left Volume"的数值是22,对应ADC ALC Gain是+24dB。我们可以逐步减小其数值，每个单位数值对应1.5dB的步进。比如我们逐步减少数值到14，对应+12dB，相较于之前衰减了12dB，播放0dBFS扫频时录音到的幅度才不失真，我们便可以将硬件的ACodec的ADC ALC Gain标定为+12dB：

```
# rk_mpi_amix_test --control "ADC ALC Left Volume" --value "14"
cmd parse result:
sound control id      : 0
control name          : ADC ALC Left Volume
control value         : 14
list controls         : 0
list contents         : 0
```

2.4.3 安防类产品音频算法调试

基于上面的音频通路测试，我们理应获得了较理想的输入信号数据，在应用集成算法之前，我们可以先确认一下测试板中是否包含了算法库和AI VQE的JSON配置参数，释放的SDK中它们默认路径下的位置分别是：

```
/oem/usr/lib/librkaudio_vqe.so
/oem/usr/lib/librkaudio_common.so
/oem/usr/share/vqefiles/config_aivqe.json
```

如果产品还有需要AI降噪，也需要确认RKNN模型的位置是否和config_aivqe.json里model_path字段描述的地址相匹配。如：

```
    "ainr": {
      "status": "enable",
      "mode" : 1,
      "datrs_optm": 2,
      "pregain" : "float:8.00",
      "mini_gain" : "float:0.006",
      "alpha1" : "float:0.6",
      "alpha2" : "float:0.8",
      "preserve1" : "float:1.05",
      "preserve2" : "float:5.0",
      "preserve3" : "float:0.6",
      "preserve4" : "float:0.95",
      "sfs_enable" : 1,
      "Sup_Freq1": 6000,
      "Sup_Freq2": -3588,
      "Sup_Energy1" : "float:100000.0",
      "Sup_Energy2" : "float:100000.0",
      "eq_enable" : 1,
      "speech_eq" : 1,
      "eq_frqs_hz" : "0  800  1500  4000  4100  4200  8000  -1  -1  -1  -1",
      "eq_gains" : "1.0  1.0  1.0   1.0   0.36  0.1786   0.1786   0   0   0   0
",
      "model_path" : "/oem/usr/share/vqefiles/rkaudio_model_ainr.rknn"
    }
```

实际rkaudio_model_ainr.rknn的存放路径也应为：

```
/oem/usr/share/vqefiles/rkaudio_model_ainr.rknn
```

然后，在应用执行之前，我们先配置一些调试相关的环境变量，以生成我们需要的信息：

```
export rt_debug_cap_file=/tmp/dbg_capin.pcm // 兼容旧版本rockit，导出算法处理前的录音数据
export rt_debug_skv_out_file=/tmp/dbg_skvout.pcm // 兼容旧版本rockit，导出AI VQE算法处理后的录音数据
export rt_debug_skv_ao_out_file=/tmp/dbg_ao_skvout.pcm // 兼容旧版本rockit，导出AO VQE算法处理后的录音数据
export rt_debug_playback_file=/tmp/dbg_playback_out.pcm // 兼容旧版本rockit，导出所有AO算法处理后的录音数据，应和数字回采信号相同
export rt_dump_skv_param=true // rockit版本通用，打印AI VQE算法运行时的参数配置
export RKAUDIO_SKV_DUMP_CONF=true // 新版本算法库使用，打印AI VQE算法运行时的参数配置
export rt_dump_skv_inout=true // 新版本rockit使用，导出AI VQE算法处理前后的录音数据，默认生成至/tmp目录下。如：/tmp/dbg-skv-in-2ch-16000.pcm和/tmp/dbg-skv-out-1ch-16000.pcm
export rt_dump_skv_ao_inout=true // 新版本rockit使用，导出AO VQE算法处理前后的录音数据，默认生成至/tmp目录下。如：/tmp/dbg-skv-ao-in-2ch-16000.pcm和/tmp/dbg-skv-ao-out-1ch-16000.pcm
```

可以使用如下命令开启AI VQE算法，且输出为1ch，所以这里'--out_ch 1'：

```
rk_mpi_ai_test --sound_card_name=hw:0,0 --device_rate 16000 --device_ch 2 --
out_rate 16000 --out_ch 1 --output /tmp --vqe_enable 1 --vqe_cfg
/oem/usr/share/vqefiles/config_aivqe.json
```

同时，可以播放本地存好的远端人声音频：

```
rk_mpi_ao_test -i /userdata/remote_voice.wav  --sound_card_name=h
w:0,0 --device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2
```

如果播放也需要走VQE算法的话，也可以加上使能AO VQE的配置参数：

```
rk_mpi_ao_test -i /userdata/remote_voice.wav  --sound_card_name=h
w:0,0 --device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2 --vqe_cf
g /oem/usr/share/vqefiles/config_aovqe.json --vqe_enable=1
```

这样，调试的时候，可以将上述的算法参数打印信息，以及AI/AO算法前后生成的数据倒出来，放到电脑上用audacity工具参看，标出时间轴，定位需要优化的地方。